

Bagging — Short Notes

What is Bagging?

Bagging = Bootstrap Aggregating

It is an **ensemble technique** that trains multiple models on different subsets of the training data and combines their predictions.

How does Bagging work?

Step 1 — Bootstrapping

- Create multiple datasets from the original dataset.
- Sample **with replacement**.
- Each dataset is used to train a separate model.

Step 2 — Train Models

- Train the same type of base model independently on each sample.

Step 3 — Aggregation

- **Classification → Majority voting**
- **Regression → Average**

Example

Original data:

$D = [1, 2, 3, 4, 5]$

Bootstrap samples could be:

$D_1 = [1, 2, 2, 4, 5]$

$D_2 = [2, 3, 3, 5, 1]$

$D_3 = [4, 4, 1, 2, 5]$

Train a separate Decision Tree on each → combine their predictions.

Why Bagging?

Bagging is especially useful for **high-variance models** such as Decision Trees.

It:

-  Reduces variance
-  Reduces overfitting
-  Improves stability
-  Usually improves prediction performance

{"learning_viz":{"type_id":"VARIANCE"}}

Types of Bagging

Technique	What is sampled?
Bagging	Rows with replacement
Pasting	Rows without replacement
Random Subspaces	Features/columns
Random Patches	Rows + features

★ Easy Memory

Bagging = Same model + different data samples + parallel training + aggregate predictions

🌲 **Random Forest is a famous example of Bagging**, using multiple Decision Trees with additional feature randomness.

🌲 Random Forest Hyperparameters — Short Notes

Random Forest hyperparameters can be divided into **3 groups**.

1. 🌲 Random Forest Parameters

Parameter Meaning

n_estimators Number of trees in the forest

max_features Number of features considered at each split

bootstrap Whether to sample rows with replacement

max_samples Number/fraction of samples used to train each tree

n_estimators

- Controls the **number of Decision Trees**.
- More trees → usually more stable predictions.
- Too many trees → more computation/time.

max_features

Controls how many features are randomly considered **at each split**.

Common values:

- "sqrt"
- "log2"
- Integer or fraction

bootstrap

- True → sampling **with replacement**

- False → sampling without replacement

Default is generally True for Random Forest.

max_samples

Controls how many training samples each tree receives.

Example:

max_samples=0.5

→ Each tree uses **50% of the training samples**.

2. 🌳 Decision Tree Parameters

Since Random Forest contains Decision Trees, tree parameters can also be tuned:

- criterion → split quality
- max_depth → maximum tree depth
- min_samples_split → minimum samples needed to split
- min_samples_leaf → minimum samples in a leaf

These mainly control **overfitting vs underfitting**.

3. ⚙️ General Parameters

n_jobs

Controls how many CPU cores are used.

n_jobs=-1

→ Use all available CPU cores → faster training.

random_state

Makes results **reproducible**.

random_state=42

class_weight

Useful for **imbalanced classification datasets**.

class_weight="balanced"

🌲 Random Forest Regressor

Most parameters are similar.

Main difference:

Classifier → Gini / Entropy / Log Loss

Regressor → MSE / MAE / other regression criteria

★ Easy Memory

Forest: n_estimators, max_features, bootstrap, max_samples

Tree: criterion, max_depth, min_samples_split, min_samples_leaf

General: n_jobs, random_state, class_weight

Hyperparameter tuning = finding the combination that gives the best generalization performance.

🌲 Random Forest Classification — Hyperparameter Tuning

1. Random Forest Performance

- Random Forest usually gives **strong performance even with default parameters**.
 - It is often a good **baseline model** before tuning.
-

2. Why Hyperparameter Tuning?

Default parameters may not give the best performance.

Tuning = trying different hyperparameter values to find a better combination.

Common parameters:

- n_estimators
 - max_features
 - max_depth
 - min_samples_split
 - min_samples_leaf
-

3. GridSearchCV 🔍

GridSearchCV tests every possible combination from the parameter grid.

Example:

```
from sklearn.model_selection import GridSearchCV
```

```
grid = {  
    'n_estimators': [100, 200],
```

```
'max_depth': [5, 10],  
'max_features': ['sqrt', 'log2']  
}
```

```
search = GridSearchCV(  
    RandomForestClassifier(),  
    grid,  
    cv=5  
)
```

```
search.fit(X_train, y_train)
```

Advantages

- ✅ Exhaustively checks all specified combinations
- ✅ Finds the best combination **within the given grid**

Disadvantage

- ❌ Can be very slow when there are many combinations.
-

4. RandomizedSearchCV 🎲

Instead of testing **every combination**, it randomly selects a specified number of combinations.

```
from sklearn.model_selection import RandomizedSearchCV
```

```
search = RandomizedSearchCV(  
    RandomForestClassifier(),  
    grid,  
    n_iter=20,  
    cv=5  
)
```

```
search.fit(X_train, y_train)
```

Advantages

- ✓ Much faster
- ✓ Good for many hyperparameters
- ✓ Often finds a near-optimal solution

Disadvantage

- ✗ May miss the absolute best combination.

🔥 GridSearchCV vs RandomizedSearchCV

	GridSearchCV	RandomizedSearchCV
Search	Every combination	Random combinations
Speed	🐢 Slower	⚡ Faster
Best when	Small search space	Large search space
Guarantee best in grid	✓ Yes	✗ No
n_iter	Not required	Controls number of trials

★ Easy Memory

GridSearch = Try EVERYTHING 🔍

RandomizedSearch = Try SOME randomly 🎲

Small dataset / few parameters → GridSearchCV

Large dataset / many parameters → RandomizedSearchCV

Out-of-Bag (OOB) Evaluation 🌳

1. What is OOB Evaluation?

OOB Evaluation is a way to evaluate a Random Forest model **without creating a separate validation set**.

It uses the data points that were **not used to train each individual tree**.

2. How does it work?

Random Forest uses **Bootstrap Sampling**:

- Data is sampled **with replacement** for each tree.
- Some rows may be selected multiple times.
- Some rows are not selected at all.
- Approximately **37% of the training samples** are left out for each tree.
- These are called **Out-of-Bag (OOB) samples**.

3. Why is OOB useful?

The OOB samples were **not seen by that tree during training**, so they can act like a **validation set**.

Therefore:

Training data → Bootstrap → Train trees + OOB samples → OOB evaluation

- ✓ No separate validation set required
- ✓ Makes better use of available training data
- ✓ Gives an estimate of generalization performance

4. Scikit-learn Example

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf = RandomForestClassifier(  
    n_estimators=100,  
    oob_score=True,  
    random_state=42  
)
```

```
rf.fit(X_train, y_train)
```

```
print(rf.oob_score_)
```

`oob_score_` → **OOB evaluation score**.

★ Easy to Remember

OOB = Data left out of a tree's bootstrap sample → used to test that tree/model.

37% rule: On average, about **36.8% ≈ 37%** of unique training observations are OOB for a bootstrap sample.

Feature Importance 🍀

1. What is Feature Importance?

Feature Importance tells us **which features (columns) are most useful for making predictions**.

For example, for loan approval:

- 💰 Income → high importance
- 📊 Credit score → high importance
- 🎂 Age → medium importance
- 🏠 Favorite color → low importance

It helps us understand **why the model makes a prediction**.

2. Why is it useful?

Feature importance is mainly used for:

-  **Model interpretation** — understand what the model relies on.
 -  **Feature selection** — remove less useful features.
 -  **Reduce training time** — fewer features can make training faster.
 -  **Remove irrelevant features** — potentially improve generalization.
-

3. Decision Tree Feature Importance

A Decision Tree gives more importance to a feature when its splits produce a **large reduction in impurity**.

Impurity can be measured using:

- **Gini Impurity**
- **Entropy**

A simplified idea is:

More impurity reduction → More feature importance

The importance values are normalized, so:

$$\sum Feature\ Importance = 1$$

Example:

Feature	Importance
Income	0.50
Credit Score	0.30
Age	0.15
Location	0.05

So **Income is the most important feature**.

4. Random Forest Feature Importance 🌲 🌲 🌲

Random Forest contains many Decision Trees.

Process:

Each tree → calculates feature importance → Average across all trees → Final importance

Therefore, Random Forest generally gives a more stable importance estimate than a single Decision Tree.

5. Scikit-learn Implementation

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf = RandomForestClassifier(random_state=42)
```

```
rf.fit(X_train, y_train)
```

```
print(rf.feature_importances_)
```

feature_importances_ gives the importance value of **each feature**.

To match them with column names:

```
for feature, importance in zip(X.columns, rf.feature_importances_):
```

```
    print(feature, importance)
```

6. ⚠ Important Limitation

Impurity-based feature importance can be misleading for high-cardinality features.

High cardinality = feature has **many unique values**.

Example:

- Age → relatively few possible values
- Customer_ID → thousands of unique values

A tree may give Customer_ID artificially high importance even though it isn't genuinely useful.

👉 In such cases, **Permutation Importance** is often preferred.

★ Easy Memory Trick

Feature Importance = Which features does the model rely on the most?

Decision Tree: impurity reduction

Random Forest: average importance across trees

High-cardinality features: consider permutation importance
